

Date:

## Prime Number Validation for RSA Key Generation

Ex.No:1-A

**AIM:**

**PROCEDURE:**

**CODE:**

```
def isPrime(n):
    if n<=1:
        return False
    for i in range(2,int(n**0.5)+1):
        if n%i==0:
            return False
    return True
p=int(input("Enter the first number(p):"))
q=int(input("Enter the second number(q):"))
if isPrime(p):
    print(p,"is a Prime Number.")
```

```

else:
    print(p,"is NOT a prime Number.")
if isPrime(q):
    print(q,"is a Prime Number.")
else:
    print(q,"is NOT a prime Number.")
if isPrime(p) and isPrime(q):
    print("\nBoth numbers are prime.")
    print("Proceed with RSA Key Generation.")
else:
    print("\nInvalid Prime Number(s).")
    print("RSA Key Generation Aborted.")

```

### OUTPUT:

```

Enter the first number (p):17
Enter the second number (q):19
17 is a Prime Number.
19 is a Prime Number.

Both numbers are prime.
Proceed with RSA Key Generation.
|

```

### RESULT:

Date:

## Secure Communication Key Validation

Ex.No: 1-B

**AIM:**

**PROCEDURE:**

**CODE:**

```
def find_gcd(a,b):  
    if a<b:  
        a,b=b,a  
    print("\nSteps of Euclidean Algorithm:")  
    while b!=0:  
        remainder=a % b  
        print(f" {a} % {b}=Quotient{a/b},Remainder{remainder}")  
        a=b  
        b=remainder  
    return a
```

```

print("=== Secure Communication Key Validation ===")
parameter1 = int(input("Enter the first encryption parameter: "))
parameter2 = int(input("Enter the second encryption parameter: "))
gcd=find_gcd(parameter1,parameter2)
print("\nGreatest Common Divisor (GCD) =", gcd)
if gcd ==1:
    print("\nThe encryption parameters are COPRIME.")
    print("Secure communication can be established.")
else:
    print("\nThe encryption parameters are NOT COPRIME.")
    print("Communication setup rejected.")
    print("Choose different encryption parameters.")

```

## OUTPUT:

```

=== Secure Communication Key Validation ===
Enter the first encryption parameter: 35
Enter the second encryption parameter: 12

Steps of Euclidean Algorithm:
35%12=Quotient2.9166666666666665,Remainder11
12%11=Quotient1.0909090909090908,Remainder1
11%1=Quotient11.0,Remainder0

Greatest Common Divisor (GCD) = 1

The encryption parameters are COPRIME.
Secure communication can be established.

```

## RESULT:

Date:

## Secure Message Encryption Using Modular Arithmetic

Ex.No: 1-C

**AIM:**

**ALGORITHM:**

**CODE:**

```
print("=====")
print("    MODULAR ARITHMETIC CALCULATOR")
print("=====")
a = int(input("Enter the First Integer (a): "))
b = int(input("Enter the Second Integer (b): "))
m = int(input("Enter the Modulus (m): "))
if m <= 0:
    print("Modulus must be greater than 0.")
else:
    while True:
        print("\n===== MENU =====")
```

```

print("1. Modular Addition")
print("2. Modular Subtraction")
print("3. Modular Multiplication")
print("4. Modular Exponentiation")
print("5. Exit")
choice = int(input("Enter your choice (1-5): "))
if choice == 1:
    result = (a + b) % m
    print("\nModular Addition")
    print(f"({a} + {b}) mod {m} = {result}")
elif choice == 2:
    result = (a - b) % m
    print("\nModular Subtraction")
    print(f"({a} - {b}) mod {m} = {result}")
elif choice == 3:
    result = (a * b) % m
    print("\nModular Multiplication")
    print(f"({a} × {b}) mod {m} = {result}")
elif choice == 4:
    result = pow(a, b, m)
    print("\nModular Exponentiation")
    print(f"({a}^{{b}}) mod {m} = {result}")
elif choice == 5:
    print("\nExiting the Program...")
    break
else:
    print("\nInvalid Choice! Please select between 1 and 5.")

```

## OUTPUT:

```

=====
MODULAR ARITHMETIC CALCULATOR
=====
Enter the First Integer (a): 15
Enter the Second Integer (b): 7
Enter the Modulus (m): 11

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 1

Modular Addition
(15 + 7) mod 11 = 0

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 3

```

```
Modular Multiplication
(15 × 7) mod 11 = 6

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 4

Modular Exponentiation
(15^7) mod 11 = 5

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 5

Exiting the Program...
```

**RESULT:**

**Date:**

## **RSA Prime Number Validation Using Fermat's Little Theorem**

**Ex.No: 1.D**

**AIM:**

**ALGORITHM:**

**CODE:**

```
print("=====")
print("  RSA Prime Validation using Fermat's Test")
print("=====")
a = int(input("Enter the value of a: "))
p = int(input("Enter the candidate prime number (p): "))
if p <= 1:
    print("\nInvalid Input!")
    print("The candidate number p must be greater than 1.")
else:
    result = pow(a, p - 1, p)
    print("\nResult of Fermat Test")
    print(f'({a}^{p-1}) mod {p} = {result}')
    if result == 1:
        print("\nThe number satisfies Fermat's Little Theorem.")
        print("It is a probable prime.")
```

```
    print("Candidate is suitable for further RSA key generation.")
else:
    print("\nThe number does not satisfy Fermat's Little Theorem.")
    print("It is composite.")
    print("Candidate is NOT suitable for RSA key generation.")
```

## OUTPUT:

```
=====
    RSA Prime Validation using Fermat's Test
=====
Enter the value of a: 2
Enter the candidate prime number (p): 17

Result of Fermat Test
(2^16) mod 17 = 1

The number satisfies Fermat's Little Theorem.
It is a probable prime.
Candidate is suitable for further RSA key generation.

-----

=====
    RSA Prime Validation using Fermat's Test
=====
Enter the value of a: 2
Enter the candidate prime number (p): 15

Result of Fermat Test
(2^14) mod 15 = 4

The number does not satisfy Fermat's Little Theorem.
It is composite.
Candidate is NOT suitable for RSA key generation.
```

## RESULT:

